B.S. IN COMPUTER ENGINEERING · 3RD YEAR

DISTRIBUTED SYSTEMS LAB PRACTICE 2: gRPC IN .NET 10

Comprehensive End-to-End Guide: IDL Contract Design, In-Memory Multithreaded Concurrency, and Distributed Architecture in C#

Course / Subject:	Distributed Systems / Distributed Computing
Academic Context:	Higher Technical School of Computer Engineering (Spain)
Platform & Language:	.NET 10.0 SDK & C# 13
Middleware / IPC:	gRPC (Google Remote Procedure Call) over HTTP/2 (h2c)
Contract Format:	Protocol Buffers v3 (proto3 IDL)
Concurrent Pattern:	Producer-Consumer with Bounded In-Memory Buffer
Synchronization:	SemaphoreSlim, ConcurrentQueue, ConcurrentDictionary, lock

Official Lab Manual & Oral Exam Defense Guide

Structured with academic rigor to enable students to build the system from scratch, comprehend all theoretical trade-offs, and ace their evaluation.

📋 **Table of Contents**

1. Theoretical Foundations of Distributed Systems
2. Overall System Architecture
3. Building the Project from Scratch (CLI)
4. The Communication Contract (IDL Protobuf)
5. Project Configuration (.csproj Files)
6. The Server (Restaurante): Concurrency & State
7. The Consumer Agent (Repartidor): Worker Loop
8. The Client Application (Cliente): Interactive UI
9. Execution Guide & Edge Test Cases
10. Bank of 10 Typical Oral Exam Questions
11. Debugging Cheatsheet & Troubleshooting
12. Conclusion & Key Engineering Takeaways

1. Theoretical Foundations of Distributed Systems

To achieve the highest grade in a 3rd-year European Computer Engineering program, it is never enough to simply "make the code compile". You must understand the conceptual models underlying modern distributed middleware and be able to justify every architectural decision before the evaluation tribunal.

1.1 What is a Remote Procedure Call (RPC)?

In a classical centralized program, a function is invoked via a *Local Procedure Call* (LPC): arguments are pushed onto the call stack or processor registers, and the CPU program counter jumps to a memory address within the same virtual address space.

In a **Distributed System**, processes run in isolated address spaces across separate operating system processes or distinct physical machines connected over a network. The goal of **RPC (Remote Procedure Call)**, originally formulated by Birrell and Nelson in 1984, is to provide **location transparency**: allowing remote function execution to appear syntactically identical to a local C# method call.

> 💡 **The Location Transparency Principle and Its Physical Limits**
> While RPC achieves *syntactic transparency*, classical theory (Deutsch's 8 Fallacies of Distributed Computing) proves that **semantic transparency is impossible**:
> • A local call never fails due to router congestion or network cable disconnects.
> • An RPC call can suffer from unpredictable latency, partial partitions, and remote server crashes.
> Consequently, gRPC mandates asynchronous signatures (`Task`), cancellation tokens, and explicit distributed exception handling via `RpcException` .

1.2 Anatomy of an RPC Call: Stubs and Skeletons

To shield developers from raw TCP sockets, byte stream parsing, and endianness handling, the RPC middleware automatically synthesizes two intermediaries from an IDL (*Interface Definition Language*) specification:

- **Stub (Client Side):** Acts as a local surrogate or proxy for the remote service. When client code calls `client.HacerPedidoAsync(request)` , the Stub intercepts the invocation, serializes parameters into a structured binary frame (**Marshalling**), and transmits it over HTTP/2.
- **Skeleton / Base Service (Server Side):** Receives the inbound network frame, reconstructs the native C# domain objects (**Unmarshalling**), dispatches the execution to your custom method in `RestauranteService` , and packages the return value back across the wire.

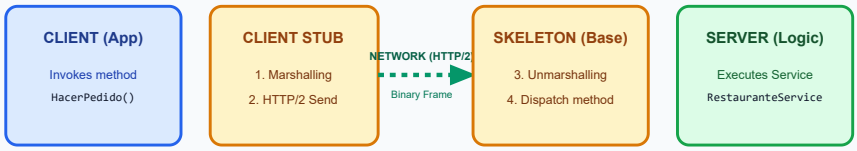


Figure 1.1: Distributed RPC invocation flow through Client Stubs and Server Skeletons.

1.3 Why gRPC and Protocol Buffers over REST and JSON?

While general web applications traditionally rely on REST over HTTP/1.1 with human-readable JSON, high-throughput distributed microservice architectures (e.g., Google, Netflix, Uber) favor gRPC for critical performance reasons:

Technical Dimension	REST Architecture with JSON	gRPC with Protocol Buffers
Message Payload Format	Structured plain text (UTF-8). Inefficient in wire size and CPU.	**Compact binary format** encoded with Varints and field tag delimiters.
Contract Definition	Optional or late (OpenAPI/Swagger). Prone to runtime drift.	**Strict and mandatory** via IDL (`.proto`). Checked at compile time.
Transport Protocol	HTTP/1.1 (or unoptimized HTTP/2). Multiple TCP connections.	**Pure HTTP/2** with multiplexed streams over a single persistent TCP connection.
CPU Serialization Cost	High: CPU must parse strings, quotes, brackets, and convert numbers to text.	**Negligible:** Direct byte offsets in RAM. Up to 7-10x faster.
Streaming Capabilities	Complex (requires WebSockets or Server-Sent Events).	**4 native streaming patterns:** Unary, Client Streaming, Server Streaming, and Bidirectional.

🔍 **Visual Example: Network Payload Overhead Comparison**

To transmit an order with `id_pedido = 1` and `distancia = 5`:
• **In JSON (REST):** `{"id_pedido":1,"distancia":5}` requires **32 bytes** in plain ASCII text.
• **In Protobuf:** Encoded into exactly **4 hex bytes**: `08 01 10 05` (Tag 1 = 1, Tag 2 = 5).
Result: An **87.5% reduction in bandwidth consumption** on every single RPC call!

1.4 Concurrency in a Stateful Distributed Service

In an in-memory stateful gRPC server, multiple network threads execute incoming requests concurrently. This creates two classic concurrency hazards:

1. **Compound Race Condition (Check-Then-Act):** If two clients submit an order when the queue has 9 of 10 slots filled, both threads might check `Count < 10` at the same time and both enqueue, violating the invariant and exceeding capacity.
2. **Thread Starvation via Busy-Waiting:** If the delivery agent constantly polled the queue in a tight loop, it would burn 100% CPU core capacity and exhaust the thread pool.

🎓 **Exam Question: Why avoid 'Thread.Sleep' or a polling 'while' loop?**

A loop like `while (queue.IsEmpty) {{ Thread.Sleep(100); }}` holds an OS thread hostage from ASP.NET Core's thread pool.
Instead, we use an **asynchronous counting semaphore (`SemaphoreSlim`)**: the delivery worker yields its thread non-blockingly via `await _readyOrdersSemaphore.WaitAsync()`, allowing Kestrel to repurpose that thread for other clients. When an order arrives, `Release()` immediately awakes the worker reactively.

2. Overall System Architecture

The system is divided into three decoupled processes communicating in real time over TCP port 5000 using HTTP/2:

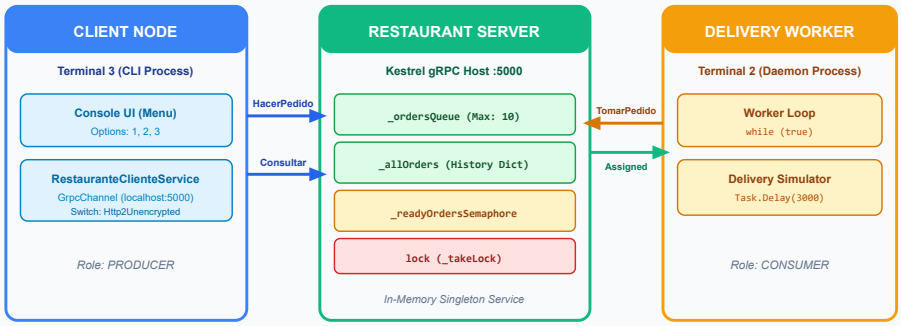


Figure 2.1: Physical architecture across the three processes and gRPC communication channels over HTTP/2.

2.1 The Producer-Consumer Pattern with a Bounded Buffer

The core algorithmic execution adheres to canonical concurrent systems theory:

1. **The Producer (Client):** Generates order requests via `HacerPedido`. It verifies that queue capacity is not exceeded (`MaxQueueCapacity = 10`). When the queue is full, the server rejects the order by returning gRPC error `StatusCode.ResourceExhausted`.
2. **The Bounded Buffer (Restaurant):** Encapsulates a thread-safe `ConcurrentQueue<Pedido>` paired with an asynchronous counting semaphore (`SemaphoreSlim`) initialized to 0 permits.
3. **The Consumer (Delivery Worker):** Requests orders by invoking `TomarPedidoParaRepartir`. If no orders exist, it awaits a permit from the semaphore. When a client enqueues an order, `_readyOrdersSemaphore.Release()` is called, awakening the worker immediately.

3. Building the Project from Scratch with the .NET CLI

To construct this entire system from an empty directory using only the command line (e.g., during a practical lab exam), execute the following commands in sequence:

3.1 Step 1: Solution and Project Initialization

Terminal / Shell — CLI Initialization Commands — BASH

```
# 1. Create root directory and navigate into it
mkdir SistemaRestaurante
cd SistemaRestaurante

# 2. Create the .NET solution file
dotnet new sln -n SistemaRestaurante

# 3. Create the Web/Kestrel project for the Server
dotnet new web -n Restaurante

# 4. Create the two Console projects for the clients
dotnet new console -n Repartidor
dotnet new console -n Cliente

# 5. Attach all three projects to the solution
dotnet sln add Restaurante/Restaurante.csproj
dotnet sln add Repartidor/Repartidor.csproj
dotnet sln add Cliente/Cliente.csproj

# 6. Create the shared directory for Protocol Buffers contracts
mkdir Protos
```

3.2 Step 2: Installing gRPC NuGet Packages

Terminal / Shell — NuGet Package Installation Commands — BASH

```
# 1. Server package (includes Kestrel HTTP/2 engine and server-side protoc code generator):
dotnet add Restaurante package Grpc.AspNetCore

# 2. Delivery worker packages (HTTP/2 client, Protobuf runtime, and protoc tools):
dotnet add Repartidor package Grpc.Net.Client
dotnet add Repartidor package Google.Protobuf
dotnet add Repartidor package Grpc.Tools

# 3. Client packages:
dotnet add Cliente package Grpc.Net.Client
dotnet add Cliente package Google.Protobuf
dotnet add Cliente package Grpc.Tools
```

Resulting Directory Structure

Verify that the folder layout in your workspace matches the following tree structure:

```
SistemaRestaurante/
├── SistemaRestaurante.slnx (or .sln)
├── Protos/
│   └── restaurante.proto
├── Restaurante/
│   ├── Restaurante.csproj
│   ├── Program.cs
│   ├── Pedido.cs
│   └── RestauranteService.cs
├── Repartidor/
│   ├── Repartidor.csproj
│   └── Program.cs
└── Cliente/
    ├── Cliente.csproj
    ├── Program.cs
    └── RestauranteClienteService.cs
```

4. The Communication Contract: Protos/restaurante.proto

In gRPC, the `.proto` file is the **Single Source of Truth**. Both the server implementation and client callers are synthesized from this IDL specification.

Protos/restaurante.proto — Protobuf Service & Message Definitions — PROTO

```proto
syntax = "proto3";
option csharp_namespace = "Restaurante";
import "google/protobuf/empty.proto";

service RestauranteService {
    rpc HacerPedido (PedidoRequest) returns (PedidoResponse);
    rpc ConsultarEstadoPedido (ConsultaRequest) returns (EstadoResponse);
    rpc TomarPedidoParaRepartir (google.protobuf.Empty) returns (PedidoResponse);
    rpc ConsultarPlatosPedido (ConsultaRequest) returns (PedidoResponse);
}

message PedidoRequest {
    repeated PlatoPedido platos = 1;
    int32 distancia = 2;
}

message PlatoPedido {
    string nombre = 1;
    int32 cantidad = 2;
}

message PedidoResponse {
    int32 id_pedido = 1;
    string mensaje = 2;
}

message ConsultaRequest {
    int32 id_pedido = 1;
}

message EstadoResponse {
    string estado = 1;
    string mensaje = 2;
    int32 tiempo_estimado = 3;
}
```

4.1 Line-by-Line Breakdown of the Protobuf Specification

- `syntax = "proto3";` (Line 1): Specifies version 3 of Protocol Buffers syntax. In `proto3`, fields are optional by default and the legacy `required` keyword is omitted, ensuring seamless forward and backward compatibility.
- `option csharp_namespace = "Restaurante";` (Line 2): Determines the C# namespace where all auto-generated classes will reside.
- `import "google/protobuf/empty.proto";` (Line 3): In gRPC, every RPC method must take exactly one message parameter and return one message. Methods cannot return `void`. Google provides the standard `google.protobuf.Empty` type for parameterless methods.
- **Service Definition `RestauranteService` (Lines 5-10):** Declares 4 unary RPC procedures:
  1. `HacerPedido (PedidoRequest) returns (PedidoResponse);`: Client submits dish items and delivery distance; server returns the assigned unique order ID.
  2. `ConsultarEstadoPedido (ConsultaRequest) returns (EstadoResponse);`: Queries current order lifecycle status given its ID.
  3. `TomarPedidoParaRepartir (google.protobuf.Empty) returns (PedidoResponse);`: Invoked by delivery worker to claim the next pending order.
  4. `ConsultarPlatosPedido (ConsultaRequest) returns (PedidoResponse);`: Returns the formatted list of dishes for a given order ID.
- **Collections with `repeated` (Line 13):** `repeated PlatoPedido platos = 1;` generates a strongly typed collection in C# (`Google.Protobuf.Collections.RepeatedField<PlatoPedido>`).
- **Field Tags (`= 1, = 2`):**

> 🎓 **Exam Question: What do the numbers '= 1' and '= 2' signify in Protobuf fields?**
>
> They are **NOT default values or variable initializers**! They are **unique binary field tags**. On the wire, gRPC does not transmit field name strings like 'distancia'; it sends the integer tag `2` encoded in a single byte. Therefore, **field numbers must never be modified once a service is deployed**, as changing them breaks wire compatibility with existing clients.

5. Project Configuration (.csproj Files)

Protobuf code generation is coordinated through MSBuild targets provided by the `Grpc.Tools` package.

5.1 Server Configuration: Restaurante.csproj

Restaurante/Restaurante.csproj — gRPC Server Project File — XML

```xml
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Grpc.AspNetCore" Version="2.84.0" />
  </ItemGroup>

  <ItemGroup>
    <Protobuf Include="..\Protos\restaurante.proto" GrpcServices="Server" />
  </ItemGroup>

</Project>
```

5.2 Delivery Worker Configuration: Repartidor.csproj

Repartidor/Repartidor.csproj — Delivery Worker (Client) Project File — XML

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Google.Protobuf" Version="3.36.2" />
    <PackageReference Include="Grpc.Net.Client" Version="2.84.0" />
    <PackageReference Include="Grpc.Tools" Version="2.84.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
  </ItemGroup>

  <ItemGroup>
    <Protobuf Include="..\Protos\restaurante.proto" GrpcServices="Client" />
  </ItemGroup>

</Project>
```

5.3 Interactive Client Configuration: Cliente.csproj

Cliente/Cliente.csproj — Interactive Client Project File — XML

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Google.Protobuf" Version="3.36.2" />
    <PackageReference Include="Grpc.Net.Client" Version="2.84.0" />
    <PackageReference Include="Grpc.Tools" Version="2.84.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
  </ItemGroup>

  <ItemGroup>
    <Protobuf Include="..\Protos\restaurante.proto" GrpcServices="Client" />
  </ItemGroup>

</Project>
```

💡 **The Technical Rationale: 'GrpcServices=Server' vs 'GrpcServices=Client'**

During `dotnet build`, the MSBuild task runs the native Google compiler (`protoc`):

- `GrpcServices="Server"` : Generates the abstract base class `RestauranteServiceBase` , containing virtual methods for the server to `override` with business logic.
- `GrpcServices="Client"` : Generates the concrete client stub `RestauranteServiceClient` , providing asynchronous wire invocation methods (`HacerPedidoAsync` , etc.).

Omitting or swapping this attribute causes build errors because the required classes will not exist in the assembly.

6. The Server Implementation (Restaurante)

The server manages order state across their complete lifecycle, configuring Kestrel HTTP/2 hosting and coordinating concurrent multithreaded synchronization.

6.1 Internal Domain Model: Restaurante/Pedido.cs

Restaurante/Pedido.cs — Internal Order Domain Entity — CSHARP

```csharp
using System.Collections.Generic;
using Restaurante;

namespace RestauranteApp
{
    public class Pedido
    {
        public int Id { get; set; }
        public List<PlatoPedido> Platos { get; set; } = new List<PlatoPedido>();
        public int Distancia { get; set; }
        public string Estado { get; set; } = "En preparacion";
    }
}
```

6.2 Host Setup and Kestrel Configuration: Restaurante/Program.cs

Restaurante/Program.cs — Kestrel Host & Dependency Injection Setup — CSHARP

```csharp
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Server.Kestrel.Core;
using Microsoft.Extensions.DependencyInjection;
using System;
using RestauranteApp;

var builder = WebApplication.CreateBuilder(args);

// Configurar Kestrel para permitir HTTP/2 sin TLS
builder.WebHost.ConfigureKestrel(options =>
{
    // Configurar el servidor para escuchar en HTTP/2 sin TLS en el puerto 5000
    options.ListenLocalhost(5000, o => o.Protocols = HttpProtocols.Http2);
});

// Agregar servicios gRPC
builder.Services.AddGrpc();

// Registrar RestauranteService como Singleton para mantener el estado
builder.Services.AddSingleton<RestauranteService>();

var app = builder.Build();

// Registra el servicio gRPC para que pueda recibir y gestionar invocaciones remotas.
app.MapGrpcService<RestauranteService>();

// Configurar una página de inicio simple
app.MapGet("/", () => "Servidor gRPC del Restaurante ejecutándose ... ");

Console.WriteLine("Iniciando servidor gRPC del Restaurante ... ");
Console.WriteLine("Presiona Ctrl+C para salir");

app.Run();
```

Deep Architectural Breakdown of Host Configuration

- **HTTP/2 Without TLS (Lines 10-15):** gRPC standard specification expects HTTP/2 over TLS negotiated via ALPN. In local development environments without SSL certificates, Kestrel must be explicitly instructed to accept **h2c (HTTP/2 Cleartext)** on port 5000: `options.ListenLocalhost(5000, o ⇒ o.Protocols = HttpProtocols.Http2);`.
- **The Mandatory Singleton Lifetime (Line 21):** `builder.Services.AddSingleton<RestauranteService>();`

> 🎓 **Crucial Exam Question: Why Register RestauranteService as a Singleton?**
> By default, ASP.NET Core gRPC registers services with a **Transient** lifecycle (a brand-new service instance is instantiated per incoming RPC call and garbage collected upon completion).
> If `RestauranteService` were Transient, each invocation of `HacerPedido` or `TomarPedidoParaRepartir` would instantiate a new empty queue and dictionary. All orders would vanish immediately!
> Registering as a **Singleton** guarantees that the IoC container keeps a **single shared instance in memory** throughout the server's lifecycle, allowing all concurrent clients and delivery workers to share the same queues, dictionaries, and semaphores.

- **Service Routing (Line 26):** `app.MapGrpcService<RestauranteService>();` connects Kestrel's HTTP/2 pipeline to our service class endpoints.

6.3 Business Logic and Concurrency: Restaurante/RestauranteService.cs

The full server implementation handling thread safety, semaphores, and error dispatching:

Restaurante/RestauranteService.cs — gRPC Service with Concurrency Control — CSHARP

```csharp
using Grpc.Core;
using System.Collections.Concurrent;
using System.Threading;
using System.Threading.Tasks;
using System.Linq;
using Google.Protobuf.WellKnownTypes;
using Restaurante;
using Microsoft.Extensions.Logging;

namespace RestauranteApp
{
    public class RestauranteService : Restaurante.RestauranteService.RestauranteServiceBase
    {
        private readonly ILogger<RestauranteService> _logger;
        private readonly ConcurrentQueue<Pedido> _ordersQueue;
        // Añadimos un diccionario para recordar todos los pedidos históricos,
        // ya que la cola los elimina cuando el repartidor los toma.
        private readonly ConcurrentDictionary<int, Pedido> _allOrders;
        private const int MaxQueueCapacity = 10;
        private readonly SemaphoreSlim _readyOrdersSemaphore;
        private readonly object _takeLock;
        private int _orderID;

        public RestauranteService(ILogger<RestauranteService> logger)
        {
            _logger = logger;
            _ordersQueue = new ConcurrentQueue<Pedido>();
            _allOrders = new ConcurrentDictionary<int, Pedido>();
            _readyOrdersSemaphore = new SemaphoreSlim(0);
            _takeLock = new object();
            _orderID = 1;
        }

        public override Task<PedidoResponse> HacerPedido(PedidoRequest request, ServerCallContext context)
        {
            lock (_takeLock) // lock the access to the queue to prevent race conditions
            {
                if (_ordersQueue.Count < MaxQueueCapacity) // checks if the queue is full
                {
                    var pedido = new Pedido // creates a new pedido with the given data
                    {
                        Id = _orderID,
                        Distancia = request.Distancia,
                        Platos = request.Platos.ToList(),
                        Estado = "En preparacion" // sets the initial state of the pedido to "En preparacion"
                    };

                    _ordersQueue.Enqueue(pedido); // adds the pedido to the queue
                    _allOrders.TryAdd(pedido.Id, pedido); // adds the pedido to the dictionary

                    _logger.LogInformation($"Recibido un nuevo pedido con ID {_orderID}");
                    _orderID++;

                    _readyOrdersSemaphore.Release(); // releases the semaphore to indicate that there is a new order

                    return Task.FromResult(new PedidoResponse // returns the response to the client
                    {
                        IdPedido = pedido.Id,
                        Mensaje = $"Pedido {pedido.Id} recibido, tiempo estimado: {pedido.Distancia * 5} mins"
                    });
                }
            }

            throw new RpcException(new Status(StatusCode.ResourceExhausted, "La cola está llena"));
        }

        public override Task<EstadoResponse> ConsultarEstadoPedido(ConsultaRequest request, ServerCallContext context)
        {
            // Buscamos en el historial global en lugar de la cola,
            // porque si el repartidor lo ha tomado, ya no está en la cola!
            if (_allOrders.TryGetValue(request.IdPedido, out var pedido)) // tries to get the pedido from the dictionary
            {
                return Task.FromResult(new EstadoResponse // returns the response to the client
                {
                    Estado = pedido.Estado,
                    Mensaje = $"Su pedido está {pedido.Estado}",
                    TiempoEstimado = pedido.Distancia * 5 // calculates the estimated time for the delivery
                });
            }
            else
            {
                throw new RpcException(new Status(StatusCode.NotFound, "El pedido no existe")); // throws an exception if the pedido is not found in the di
            }
        }

        public override async Task<PedidoResponse> TomarPedidoParaRepartir(Empty request, ServerCallContext context)
        {
            await _readyOrdersSemaphore.WaitAsync();
```

```
            lock (_takeLock)
            {
                if (_ordersQueue.TryDequeue(out var pedido))
                {
                    pedido.Estado = "En reparto";
                    return new PedidoResponse
                    {
                        IdPedido = pedido.Id,
                        Mensaje = "Pedido en reparto"
                    };
                }
            }

            throw new RpcException(new Status(StatusCode.NotFound, "No hay pedidos disponibles"));
        }

        public override Task<PedidoResponse> ConsultarPlatosPedido(ConsultaRequest request, ServerCallContext context)
        {
            if (_allOrders.TryGetValue(request.IdPedido, out var pedido))
            {
                var mensaje = string.Join(", ", pedido.Platos.Select(p => $"{p.Cantidad}x {p.Nombre}"));
                return Task.FromResult(new PedidoResponse {
                    IdPedido = pedido.Id,
                    Mensaje = mensaje
                });
            }
            else
            {
                throw new RpcException(new Status(StatusCode.NotFound, "El pedido no existe"));
            }
        }
    }
}
```

Exhaustive Concurrency Breakdown in RestauranteService

1. Data Structures and Synchronization Primitives (Lines 15-22):

- `ConcurrentQueue<Pedido> _ordersQueue;` : Thread-safe FIFO queue storing orders waiting to be claimed by delivery workers.
- `ConcurrentDictionary<int, Pedido> _allOrders;` :

> 💡 **The Need for the Historical ConcurrentDictionary**
> When a delivery worker invokes `TomarPedidoParaRepartir`, the order is **dequeued** (removed from `_ordersQueue`). If a client subsequently calls `ConsultarEstadoPedido(id)` and we only had the queue, the server would return 'Not Found'!
> `_allOrders` functions as the **persistent in-memory history log**, keeping the object accessible in $O(1)$ constant time so its lifecycle state ('En reparto') can be monitored at all times.

- `SemaphoreSlim _readyOrdersSemaphore = new SemaphoreSlim(0);` : Counting semaphore initialized to **0 permits**. When no orders exist, any delivery worker calling `WaitAsync()` suspends asynchronously. When an order is enqueued, `Release()` increments the permit count, immediately awakening a worker.
- `object _takeLock = new object();` :

> ⚠️ **Why is 'lock' necessary if ConcurrentQueue is already thread-safe?**
> Concurrent collections only guarantee atomicity for isolated single operations (`Enqueue` or `TryDequeue`).
> However, the sequence: `if (_ordersQueue.Count < MaxQueueCapacity) {{ _ordersQueue.Enqueue( ... ); }}` is a compound **Check-Then-Act** operation. If two threads read `Count = 9` simultaneously, both pass the `if` condition and enqueue, pushing the count to 11. The `lock (_takeLock)` ensures the check and enqueue occur as an indivisible atomic critical section.

2. RPC Method Analysis:

- **`HacerPedido` (Lines 34-65):** Acquires `lock (_takeLock)`, checks if `_ordersQueue.Count < MaxQueueCapacity`. If capacity exists, creates the order, adds it to both the queue and the historical dictionary, calls `_readyOrdersSemaphore.Release()`, and returns the response. If the queue is saturated (10 orders), throws `RpcException(new Status(StatusCode.ResourceExhausted, "La cola está llena"))`.
- **`ConsultarEstadoPedido` (Lines 67-84):** Looks up the order in `_allOrders` via `TryGetValue`. If found, returns the status and estimated delivery time (`distancia * 5` mins). If absent, throws an RPC error with `StatusCode.NotFound`.
- **`TomarPedidoParaRepartir` (Lines 86-104):** Calls `await _readyOrdersSemaphore.WaitAsync();`, suspending without blocking OS threads if no orders are available. Once a permit is granted, locks `_takeLock`, calls `_ordersQueue.TryDequeue`, updates the status to "En reparto", and returns the order.
- **`ConsultarPlatosPedido` (Lines 106-120):** Retrieves the order from the dictionary and formats the list of dishes into a readable string using `string.Join`.

7. The Consumer Agent (Repartidor)

The delivery agent is a gRPC client operating as a **background daemon worker**. It continuously consumes pending orders in an infinite loop and simulates physical transit.

Repartidor/Program.cs — Asynchronous Worker Loop in Delivery Agent CSHARP

```csharp
using System;
using System.Net.Http;
using System.Threading.Tasks;
using Grpc.Net.Client;
using Google.Protobuf.WellKnownTypes;
using Restaurante;
using Grpc.Core;

namespace RepartidorApp
{
    class Program
    {
        static async Task Main(string[] args)
        {
            var httpHandler = new HttpClientHandler
            {
                ServerCertificateCustomValidationCallback = HttpClientHandler.DangerousAcceptAnyServerCertificateValidator
            };

            var channel = GrpcChannel.ForAddress("http://localhost:5000", new GrpcChannelOptions { HttpHandler = httpHandler });
            var client = new RestauranteService.RestauranteServiceClient(channel);

            Console.WriteLine("Repartidor iniciado. Esperando pedidos para repartir...");

            while (true)
            {
                try
                {
                    var response = await client.TomarPedidoParaRepartirAsync(new Empty());
                    Console.WriteLine($"[Asignado] {response.Mensaje} (ID: {response.IdPedido})");

                    Console.WriteLine("Entregando...");
                    await Task.Delay(3000); // Simulate delivery time
                    Console.WriteLine($"[Entregado] Pedido ID: {response.IdPedido} entregado con exito.\n");
                }
                catch (RpcException ex)
                {
                    Console.WriteLine($"[Error] {ex.Status.Detail}");
                    await Task.Delay(2000); // Retry delay
                }
                catch (Exception ex)
                {
                    Console.WriteLine($"[Excepcion] No se pudo conectar: {ex.Message}");
                    await Task.Delay(3000); // Retry delay
                }
            }
        }
    }
}
```

7.1 Analysis of the Delivery Worker Operation

- **Channel Instantiation (Line 20):** `GrpcChannel.ForAddress("http://localhost:5000", ...)` opens the HTTP/2 connection to the server.
- **Infinite Loop (Line 25):** Runs a continuous `while (true)` pull loop.
- **Asynchronous Remote Call (Line 29):** Invokes `await client.TomarPedidoParaRepartirAsync(new Empty())`. Because of the server's semaphore, this call remains suspended on the server side without consuming CPU cycles until an order arrives.
- **Transit Simulation (Line 33):** `await Task.Delay(3000);` suspends execution for 3 seconds to simulate physical road transit.
- **Fault Tolerance & Retries (Lines 36-45):** If the server is offline or restarts, catches `RpcException` and general network exceptions, waiting 2-3 seconds before retrying, preventing process crashes.

8. The Client Application (Cliente)

The client application cleanly decouples gRPC communication logic from console user interface interactions.

8.1 Communication Service: Cliente/RestauranteClienteService.cs

Cliente/RestauranteClienteService.cs — Client Stub Wrapper Service — CSHARP

```csharp
using Grpc.Net.Client;
using Grpc.Core;
using System;
using System.Threading.Tasks;
using System.Collections.Generic;
using Restaurante;

namespace ClienteApp
{
    public class RestauranteClienteService
    {
        private GrpcChannel? _channel;
        private RestauranteService.RestauranteServiceClient? _client;

        public void ConnectAsync()
        {
            AppContext.SetSwitch("System.Net.Http.SocketsHttpHandler.Http2UnencryptedSupport", true);

            _channel = GrpcChannel.ForAddress("http://localhost:5000");
            _client = new RestauranteService.RestauranteServiceClient(_channel);
        }

        public async Task HacerPedidoAsync(List<PlatoPedido> platos, int distancia)
        {
            var request = new PedidoRequest { Distancia = distancia };
            request.Platos.AddRange(platos);

            try
            {
                var response = await _client!.HacerPedidoAsync(request);
                Console.WriteLine($"[Exito] Pedido ID: {response.IdPedido} - {response.Mensaje}");
            }
            catch (RpcException ex)
            {
                Console.WriteLine($"[Error] Code: {ex.StatusCode}, Mensaje: {ex.Status.Detail}");
                throw;
            }
        }

        public async Task ConsultarEstadoPedidoAsync(int idPedido)
        {
            var request = new ConsultaRequest { IdPedido = idPedido };

            try
            {
                var response = await _client!.ConsultarEstadoPedidoAsync(request);
                Console.WriteLine($"[Estado] {response.Estado}");
                Console.WriteLine($"[Mensaje] {response.Mensaje}");
                Console.WriteLine($"[Tiempo Estimado] {response.TiempoEstimado} mins");
            }
            catch (RpcException ex)
            {
                Console.WriteLine($"[Error] No se pudo consultar: {ex.Status.Detail}");
            }
        }
    }
}
```

> ⚠️ **The Security Compatibility Switch: Http2UnencryptedSupport**
>
> In line 17: `AppContext.SetSwitch("System.Net.Http.SocketsHttpHandler.Http2UnencryptedSupport", true);`
>
> **Why is this strictly required?** By default, .NET's HTTP socket handler prohibits unencrypted HTTP/2 connections over plain `http://` (h2c) for production security. Without setting this runtime flag, .NET throws an immediate `InvalidOperationException: The HTTP/2 unencrypted support switch must be set`.

8.2 Interactive Console UI: Cliente/Program.cs

Cliente/Program.cs — Interactive Console User Interface

CSHARP

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Restaurante;
using ClienteApp;

namespace Cliente
{
    class Program
    {
        static async Task Main(string[] args)
        {
            var clientService = new RestauranteClienteService();
            clientService.ConnectAsync();
            Console.WriteLine("Conectado al servidor gRPC.");

            while (true)
            {
                Console.WriteLine("\n═══ Menú ═══");
                Console.WriteLine("1. Hacer un pedido");
                Console.WriteLine("2. Consultar estado del pedido");
                Console.WriteLine("3. Salir");
                Console.Write("Opcion: ");

                var opcion = Console.ReadLine();

                // Keep asking for an option if it's invalid, without reprinting the whole menu
                while (string.IsNullOrWhiteSpace(opcion) || (opcion ≠ "1" && opcion ≠ "2" && opcion ≠ "3"))
                {
                    if (!string.IsNullOrWhiteSpace(opcion))
                    {
                        Console.WriteLine("Opción no válida.");
                    }
                    Console.Write("Opcion: ");
                    opcion = Console.ReadLine();
                }

                if (opcion == "1")
                {
                    var platos = new List<PlatoPedido>();
                    while(true)
                    {
                        Console.Write("Nombre del plato (o vacio para terminar): ");
                        var nombre = Console.ReadLine();
                        if (string.IsNullOrWhiteSpace(nombre)) break;

                        Console.Write("Cantidad: ");
                        if (int.TryParse(Console.ReadLine(), out int cantidad))
                        {
                            platos.Add(new PlatoPedido { Nombre = nombre, Cantidad = cantidad });
                        }
                    }

                    if (platos.Count > 0)
                    {
                        Console.Write("Distancia de envio: ");
                        if (int.TryParse(Console.ReadLine(), out int distancia))
                        {
                            try
                            {
                                await clientService.HacerPedidoAsync(platos, distancia);
                            }
                            catch { }
                        }
                    }
                }
                else if (opcion == "2")
                {
                    Console.Write("ID del pedido: ");
                    if (int.TryParse(Console.ReadLine(), out int id))
                    {
                        await clientService.ConsultarEstadoPedidoAsync(id);
                    }
                }
                else if (opcion == "3")
                {
                    break;
                }
            }
        }
    }
}
```

Key User Experience Highlights:

- **Robust Input Validation (Lines 28-36):** If the user enters an invalid menu option, the system warns "Opción no válida" and reprompts without redundantly redrawing the entire menu banner.
- **Dynamic Dish Composition (Lines 40-52):** Allows users to add multiple dishes dynamically until pressing Enter with an empty dish name.

- **Protobuf Collection Mapping (Lines 24-26 in ClienteService):** The native C# `List<PlatoPedido>` is appended to the Protobuf repeated field via `request.Platos.AddRange(platos)`.

9. Execution Guide & Edge Test Cases

To demonstrate distributed communication, you must launch all three projects simultaneously in **three separate terminal windows** from the solution root folder.

9.1 Strict Startup Sequence

Step 1: Start the Server (Restaurante)

In the first terminal, run:

```
dotnet run --project Restaurante
```

Expected output: `Iniciando servidor gRPC del Restaurante ...`

Step 2: Start the Delivery Worker (Repartidor)

In the second terminal, run:

```
dotnet run --project Repartidor
```

Expected output: `Repartidor iniciado. Esperando pedidos para repartir ...`

Step 3: Start the Client (Cliente)

In the third terminal, run:

```
dotnet run --project Cliente
```

Expected output: `Conectado al servidor gRPC.` followed by the interactive menu.

9.2 Practical Lab Demonstration Scenarios

Scenario 1: Standard Order Placement & Immediate Dispatch

1. In **Cliente**, choose `1`. Enter dish: "*Double Burger*", quantity: *2*. Press Enter with empty text to finish. Distance: *4*.
2. **Client response:** `[Exito] Pedido ID: 1 – Pedido 1 recibido, tiempo estimado: 20 mins`.
3. **Server log:** `info: Recibido un nuevo pedido con ID 1`.
4. **Delivery Agent response:**
   ```
   [Asignado] Pedido en reparto (ID: 1)
   Entregando...
   (waits 3 seconds)
   [Entregado] Pedido ID: 1 entregado con exito.
   ```
5. In **Cliente**, during the 3-second transit, select option `2` and query ID `1`. You will see: `[Estado] En reparto`.

Scenario 2: Bounded Buffer Saturation (Queue Full)

To demonstrate that bounded queue protection works correctly:

1. Stop the **Delivery Worker** by pressing `Ctrl + C` in its terminal window (simulating busy workers).
2. From **Cliente**, place 10 consecutive orders (IDs 2 through 11). All 10 will succeed.
3. Attempt to submit the **11th order**.
4. **Result:** The server rejects the order and the client prints:
   ```
   [Error] Code: ResourceExhausted, Mensaje: La cola está llena
   ```
5. Restart the Delivery Worker (`dotnet run --project Repartidor`). Watch it immediately drain the backlog one by one without losing any orders.

10. Bank of 10 Typical Oral Exam Questions

Below are the 10 most common conceptual questions asked by university professors during project defense evaluations, accompanied by model answers:

Q1: Why does gRPC use HTTP/2 instead of HTTP/1.1, and what advantages does it provide here?

Answer: HTTP/1.1 suffers from application-layer Head-of-Line Blocking: each request requires its own TCP connection or waits sequentially. HTTP/2 introduces **multiplexed binary bidirectional streams** over a single persistent TCP socket. This allows status queries and new orders to travel concurrently over the same connection without blocking each other, alongside HPACK header compression.

Q2: Why must RestauranteService be registered as a Singleton in Dependency Injection?

Answer: By default, ASP.NET Core registers services as Transient (instantiated per call and destroyed upon completion). Our server maintains **in-memory state** (the queue, dictionary, and semaphore). If it were Transient, each RPC invocation would run on a fresh, empty instance, causing state to be lost immediately.

Q3: Why is SemaphoreSlim used instead of a polling loop in the delivery worker?

Answer: Polling with `Thread.Sleep` wastes CPU cycles and adds artificial latency. `SemaphoreSlim.WaitAsync()` provides **reactive, non-blocking synchronization**: the delivery worker yields its thread back to the runtime thread pool and is only reactivated when a client produces an order via `Release()`.

Q4: If ConcurrentQueue is already thread-safe, why is 'lock (_takeLock)' required?

Answer: Concurrent collections only guarantee atomic safety for isolated single operations (`Enqueue` or `TryDequeue`). Checking capacity and then enqueuing (`if (_ordersQueue.Count < MaxQueueCapacity) Enqueue()`) is a compound *Check-Then-Act* operation. Two threads checking simultaneously when count is 9 would both succeed, resulting in 11 items. The `lock` makes the check-and-insert an indivisible atomic critical section.

Q5: Why do we maintain both a queue (_ordersQueue) and a dictionary (_allOrders)?

Answer: They serve distinct roles. The queue implements the dispatch buffer; when a worker claims an order, it is permanently dequeued. If a client queries an order after dispatch, the queue no longer contains it. The dictionary serves as the persistent in-memory historical repository with $O(1)$ lookups, allowing status tracking until final delivery.

Q6: What role does the GrpcServices attribute play in .csproj project files?

Answer: It instructs the `protoc` compiler which C# artifacts to synthesize. `GrpcServices="Server"` generates the abstract base service class containing virtual methods for server overrides. `GrpcServices="Client"` generates the client stub containing asynchronous remote call proxies.

Q7: Why is 'AppContext.SetSwitch("...Http2UnencryptedSupport", true)' necessary in the client?

Answer: For security reasons, .NET's native HTTP handler (`SocketsHttpHandler`) prohibits unencrypted HTTP/2 over plain `http://` by default. Setting this switch explicitly permits cleartext HTTP/2 (h2c) for local development on `http://localhost:5000`.

Q8: How does gRPC handle errors differently than traditional REST APIs?

Answer: While REST relies on generic HTTP status codes (400, 404, 500) and arbitrary body schemas, gRPC uses 16 canonical `StatusCode` values (such as `NotFound` , `ResourceExhausted` , `Unavailable`) carried in HTTP/2 trailing headers, throwing a strongly typed `RpcException` in client code.

Q9: What communication invocation semantics does this system provide?

Answer: gRPC defaults to **At-Most-Once** semantics: if a network failure occurs, the call is not automatically retried to prevent duplicating non-idempotent actions (like placing duplicate orders). Achieving *At-Least-Once* requires client retries, and *Exactly-Once* requires idempotency tokens on the server.

Q10: What streaming modes exist in gRPC and how could streaming improve this lab?

Answer: gRPC supports 4 modes: Unary (used here), Server Streaming, Client Streaming, and Bidirectional. The `ConsultarEstadoPedido` method could be upgraded to a **Server Streaming RPC**: the client subscribes once, and the server pushes live updates ("En preparación" -> "En reparto" -> "Entregado") over an open stream, eliminating manual client polling.

11. Debugging Cheatsheet & Troubleshooting

Common runtime and compilation issues encountered during development and their fixes:

Error Message / Symptom	Root Cause	Immediate Resolution
`Failed to bind to address http://localhost:5000: address already in use`	A previous Restaurant server instance is still running in the background holding port 5000.	In PowerShell, terminate the zombie process: `Stop-Process -Name "Restaurante" -Force` or inspect PID with: `Get-NetTCPConnection -LocalPort 5000`.
`The HTTP/2 unencrypted support switch must be set`	The client is connecting over `http://` without authorizing cleartext HTTP/2.	Add before creating the channel: `AppContext.SetSwitch("System.Net.Http.SocketsHttpHandler.Http2UnencryptedSupport", true);`
`Status(StatusCode="Unavailable", Detail="Failed to connect to all addresses")`	The server is not currently running or listening on the expected port.	Ensure the server is started first via `dotnet run --project Restaurante` and listening on port 5000.
`The type or namespace name 'RestauranteServiceBase' could not be found`	Protobuf file has not been compiled or `GrpcServices="Server"` is missing in the project file.	Check that `Restaurante.csproj` contains `<Protobuf Include="..\Protos estaurante.proto" GrpcServices="Server" />` and run `dotnet build`.

12. Key Engineering Competencies Acquired

Upon completing this lab, you have mastered the core pillars of enterprise distributed systems engineering:

- Formal language-agnostic interface design using **Protocol Buffers IDL**.
- High-performance binary network transport over **HTTP/2 (h2c)**.
- Thread-safe concurrent coordination with **atomic critical sections, bounded buffers, and asynchronous semaphores**.
- Distributed component lifecycle control via **Dependency Injection (Singleton)** in .NET 10.
- Canonical distributed fault handling using **gRPC Status Codes and typed exceptions**.

Distributed Systems Laboratory · Practice 2 · Engineered with academic rigor for Computer Science and Engineering degrees.